

THOR: Lightning Detection & Alert System

Eoghan Ó Bogail



Stand number: 4201
Pobalscoil Inbhear Scéine
Project Book

Comments

Abstract

This book showcases a lightning detection and alert system I have been developing since September 2024 called THOR. Rather than being contained in a single unit, different tasks are handled by different devices, which all communicate with a central server, allowing for easy modification of the system.

THOR's central server (the Hub) is designed to be run on older and lower-powered hardware, to allow it to run on devices like a Raspberry Pi or an old desktop, which is helped in part by the Hub not needing to detect anything itself. Sensors (Provider Nodes) are separate devices, allowing for easy modification of just the sensors without needing to change anything on the server.

The lightning sensor itself is a DFRobot Gravity Lightning Sensor (which uses an AMS AS3935 Franklin Lightning Sensor IC and an antenna), which can detect lightning from up to 40km away. Due to some issues getting the sensor to communicate with an Arduino Nano ESP32 over I²C, I am using an Arduino Uno to communicate with it instead, and telling the Nano about it over a UART serial connection, before it then tells the Hub about it over the MQTT protocol. In my testing, the average time between the Nano sending an alert and my computer receiving it was 739 milliseconds (see Figure 2.3)

Alerts from THOR can currently be received in two ways: a "display node" and a desktop app. The display node is a Raspberry Pi Zero 2 W connected to an e-Paper display that shows the current alerts. The desktop app, which works on Linux, macOS and Windows, runs in the background, waiting for an alert, and when there is one it notifies the user. Because alerts can be received over both the MQTT and Socket.IO protocol, creating other methods to receive alerts, like a mobile or smart watch app, should be fairly simple.

In the future, I would like THOR to support more kinds of alerts, like earthquake early warnings or low air quality. For this reason, I made THOR open source to promote developing more kinds of alerts and adding support for more sensors. While THOR can work fully offline, I have already made it possible to receive weather updates from MET Norway and weather warnings from Met Éireann.

Contents

Comments	2
Abstract	3
Contents	4
1 Introduction	5
1.1 Definitions	5
1.2 How is Lightning Detected?	6
1.3 Deciding on a Language	7
1.4 Open Source	7
2 The System	8
2.1 Hubs and Nodes	8
2.2 THOR Hub	8
2.2.1 Receiving Lightning Alerts	8
2.2.2 Getting Data from External Providers	9
2.2.3 Web Dashboard	11
2.2.4 Autodiscovery & Registration	11
2.2.5 System Requirements	12
2.2.6 MQTT Bridge	13
2.3 Lightning Node	13
2.3.1 Testing	14
2.4 Alerts App	15
2.4.1 THORICONS	16
2.5 Display Node	17
3 Conclusion	19
3.1 Acknowledgments	19
References	21

Chapter 1

Introduction

I live in rural Ireland, which means that I am no stranger to power cuts. A combination of wind and lightning during thunderstorms often leads to a power line or a tree falling, leaving my house without any power. These power cuts can cause small power surges to occur, which most devices aren't designed to deal with and will eventually stop working over time if the surges keep happening. If lightning strikes the house directly, say through a satellite dish, the surge would be in the millions of volts, which would absolutely kill the devices that are plugged in[3].

All of this means it is important to know if lightning is approaching. One of the simplest ways of knowing that lightning is nearby is by looking for the flash and listening for the thunder. This isn't very useful to me though, as I may not notice the flash and I tend to spend a lot of my time using a computer listening to YouTube or music, and I often don't check the weather forecast, so I often miss these cues.

Experiences like this got me thinking: what if I could get an alert on my computer when lightning strikes? Would there be any other use cases for such a system? Would I know about the lightning before I can hear the thunder?

And so I set out to make this system, with the goals of sensing lightning, processing that alert, and sending that alert out to various devices.

1.1 Definitions

Before continuing I would like to define some of the terms and acronyms I will be using in this booklet.

Hub refers to both the main program controlling THOR and the hardware it is running on.

You may see **Hub server** or just **server** used to refer to the hardware THOR is running on.

You May also see **Hub software** used to refer to the program running on the server.

Node refers to hardware or software that directly gives THOR data or retrieves data from THOR.

Hardware or software that retrieves data from THOR are referred to as "receiver nodes"

Hardware or software that supplies data to THOR is referred to as "provider nodes"

Provider refers to an external source of data from outside of the THOR system, such as from a weather station or the Internet. These are fetched by the **Hub**.

NOAA stands for National Oceanic and Atmospheric Assosication, and is a part of the US Government.

MQTT is a protocol for two computers to send data to each other, designed to use little bandwidth and allow

DHCP stands for Dynamic Host Configuration Protocol. It is a protocol for devices on a network to get automatically assigned an IP address instead of it being manually assigned. Unless you've changed your local IP address, you're probably using it on your home Wi-Fi right now.

e-Paper is also known as eInk.

TCP and UDP are protocols for computers to communicate with each other over a network. TCP is a one-on-one connection, with both sides acknowledging each other's messages and acknowledgements to ensure both computers know if data arrived to the other computer. UDP, on the other hand, allows a computer to send or receive data packets to or from the entire network, but the sender has no way of knowing if the packet arrived by default.

JSON is a data type similar to JavaScript objects/arrays or Python dicts/lists.

GUI stands for "Graphical User Interface".

I²C stands for Inter-Integrated Circuit. It is a communication method for integrated circuits to communicate with each other over a short distance.

1.2 How is Lighting Detected?

There are several ways of detecting lightning, for example the NOAA and NASA have an instrument called the Geostationary Lightning Mapper on two of their satellites which optically detects flashes of lightning, [5] however this would be inconvenient for home use.

Another way of detecting lightning involves detecting radio frequencies (RF). When lightning strikes, it sends out a pulse of electromagnetic energy, including the light we see as a flash[2]. Networks such as ATDNET, which was created by the Met Office in the UK and is accurate to less than 3km in western Europe, detect these pulses known as "sferics" in the frequency range of 10-14kHz (known as Very Low Frequency or VLF). It then uses triangulation to find the source of the sferic[2]. Sferics can also be picked up on simple AM radios. The lightning sensor I am using in this system, the AS3935 Franklin Lightning Sensor IC, listens to a different frequency at around 500 kHz. [6] Because this type of detection uses radio waves, detectors need to filter out interference from other devices, which may not be as big of a problem for ATDNET, but for something to be used at home its more of a concern.

1.3 Deciding on a Language

I decided to use Python as the main language for this project, simply because I'm very familiar with it. I have been using Flask, a web app framework for Python, for several years at this point, and so using Python and Flask makes building the Hub very easy for me. I didn't really consider any other languages when building this project. Of course, there is some JavaScript in the Hub for client-side code execution, but the entire rest of the project uses Python.

1.4 Open Source

I decided to make this project open source for a simple reason: I want to make it easy to extend THOR past just lightning. As I can't test various types of sensors myself,¹ I'd rather people were able to work out how to add their own sensors to the system. I also feel that the work I have done on this project is worth sharing, and I think the best way to do that is to make everything open for people to see it and learn from what I have done but also where I have gone wrong.

I published my code as a Git repository at <https://github.com/sisterbones/thor>

¹I don't know how legal it would be to create an earthquake just to test a sensor.

Chapter 2

The System

2.1 Hubs and Nodes

THOR operates on a server-client model, with the server being the Hub and the clients being the various Nodes. The Nodes connect to the Hub over a local network. For example, in THOR the lightning sensor is separated from the Hub into a separate Node. This allows for more flexibility, as now the lightning sensor isn't tethered to the Hub is running on.

Nodes connect to the hub over the network, but in the future I would like to add fallback Wi-Fi and Bluetooth Low Energy connections to keep the system robust.

2.2 THOR Hub

The THOR hub is the central server that the nodes and receivers connect to. It consists of three separate programs: a Python web app using Flask with a Socket.IO server, an MQTT to Socket.IO bridge, and a UDP auto-discovery server. The Flask app is able to run without MQTT support, which makes it easier to update when not connected to a network.

The hub receives and distributes data using Socket.IO, which is a low-latency communication protocol that mainly uses WebSockets. I picked Socket.IO because it is simple to implement on both the client and it handles reconnections automatically. The MQTT bridge allows the hub to receive alerts from Internet of Things devices.

2.2.1 Receiving Lightning Alerts

After the lightning sensor sends its data (see 2.3), the Hub must handle this data. To make it easier to add new types of alerts in the future, I created a

class called `Alert` to hold the common things an alert needs, like time, source, severity, id, and so on.

I then derived a class from that called `LightningAlert`, which takes in how far away the lightning is and sets all the other parameters automatically. The hub then checks the database to see if there's already an active lightning alert, and if it is it checks if the new alert is closer than the older ones. If so, or if there aren't any active lightning alerts, it adds the alert to the database and publishes the alert to any nodes listening for alerts.

I also created a class called `InfoAlert`, which allows me to make alerts that display as just informational rather than an actual alert. This is useful because the lightning sensor informs about too much noise, so I can use `InfoAlert` to inform the user about it without needing to send a notification.

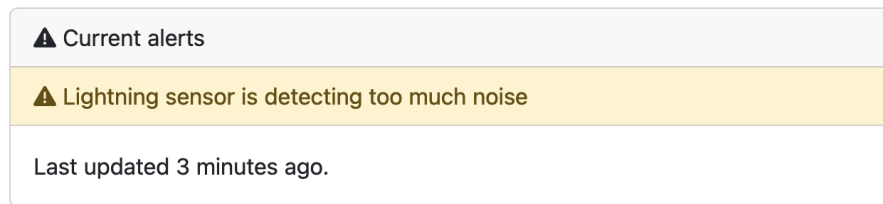


Figure 2.1: An example of `InfoAlert` in practice

2.2.2 Getting Data from External Providers

THOR supports getting weather and alert data from various sources that are external from the system. It could just be a weather station locally, or a source from the Internet. I have implemented two such sources: weather forecasts provided by MET Norway, and weather warnings from Met Éireann.

To do this, I first created a class, named `WeatherProvider`, to represent a generic weather provider, which outlines how it should return weather information. This class doesn't actually fetch any data though, so another class `CachingWeatherProvider` handles fetching and caching data from weather providers. It is good practice to cache data like this, as it avoids hammering the APIs with requests. It also includes a `User-Agent` header, not only because it's good practice, but MET Norway requires it for the `Locationforecast` API they provide[4].

```
class CachingWeatherProvider(WeatherProvider):
    def __init__(self):
        self.last_fetched_time = 0
        self.seconds_to_live = 6000 # 10 minutes
        self.cached_response = None
```

```

self.user_agent = "Thor Lightning Detection & Alert
System (in development)
https://github.com/sisterbones/thor"
self.query = {}

def fetch(self) -> dict:
    if not has_internet_connection(): # Helper function to
        check if there's internet
    return self.cached_response
    if time.time() <= self.last_fetched_time +
        self.seconds_to_live and self.cached_response: #
        Check if cache has expired
    log.debug("Serving cache")
    return self.cached_response
    try:
        data_request = requests.get(
            self.base_url,
            self.query,
            timeout=(1, 10),
            headers={
                "User-Agent": self.user_agent
            })
    except (requests.exceptions.ConnectionError,
            requests.exceptions.ReadTimeout):
        log.critical(f"Couldnt connect to {self.base_url}.")
        return self.cached_response
    self.cached_response = data_request
    self.last_fetched_time = time.time()
    return data_request

```

For the MET Norway Locationforecast API, I simply set the `base_url` parameter to the URL of the API and passed in the latitude and longitude as query parameters. It then takes the latest forecast (which would be the one for the current hour) and gets the temperature and current weather symbol (intended for showing an icon next to the weather information on Yr.no, which is MET Norway's weather forecast website), and maps the weather symbol to an icon in the Font Awesome Free set and a headline summarizing the weather, before returning the data.

Met Éireann Weather Warnings use the `Alert` class referenced in *Receiving Lightning Alerts* (2.2.1). I created a class called `MetEireannWeatherWarning` which takes data from Met Éireann's Open Data website¹. This allows me to

¹Annoyingly, the endpoint for getting weather warnings as a JSON file is broken. For example, the specifications for the JSON feed says that `type` should be something like `Rain` or `Wind`,[1] but in practice it is actually something like `orange`; `Severe`, which is data I can already determine from other fields in the JSON object. The data I want, which is the type of alert, cannot be parsed without manipulating the headline, because of which some of the code I wrote for the display node that depends on what the specs say is broken (see Figure 2.7c). On top of that, the JSON feed doesn't include line breaks for the description for some reason. In the future, it would probably be better to use their RSS feed.

publish the weather warnings like lightning alerts, and store them in the same database table.

2.2.3 Web Dashboard

To allow for easy configuration of THOR, I created a web dashboard for it (it is a Flask app after all). The dashboard shows current alerts, the weather and basic configuration options (see Figure 2.2).

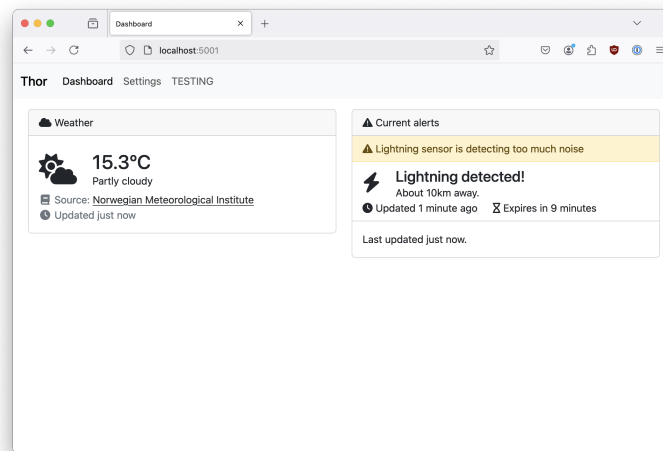


Figure 2.2: A screenshot showing the THOR Dashboard.

To speed up the UI creation process, I used Bootstrap as the UI framework. Bootstrap allows you to quickly make fluid sites with a consistent design, and it has plenty of resources online, so it was easy to get the hang of. The UI is rendered mostly server-side using Jinja templates, but there is some JavaScript being used for the "Current Alerts" and Weather cards.

The JavaScript simply connects to the Socket.IO server to receive updates about the weather and alerts, and rebuilds the alert list.

The dashboard also allows you to configure some simple options, like the location to use for Providers and which providers the THOR should use. Aside from that, there is not much else that it does at the moment.

2.2.4 UDP Autodiscovery & Node Registration

The Hub also has a second, optional script to be run alongside the main Flask app for the Hub. This allows nodes to find the Hub's IP address without needing to be told what it is, which is useful if the Hub is using DHCP to get its IP address.

The autodiscovery script implements the system laid out in ?? (??) using the built-in `socket` module in Python. It listens for UDP packets on port 51366 and if the packet starts with `thor_node_broadcast` it will respond with the sender's IP address and the port the Hub is running on.

```
# Logging code has been removed. Based on
# https://github.com/jholtmann/ip_discovery
import socket
def discovery_loop():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_address = ('', environ.get('NODE_DISCOVERY_PORT',
    51366))
    sock.bind(server_address)
    while True:
        data, address = sock.recvfrom(4096)
        data = str(data.decode('UTF-8'))
        if data.startswith('thor_node_broadcast'):
            response =
                f'thor_server_response;your_ip:{address[0]};port:{environ.get("PORT",
                "8467")}'
            sent = sock.sendto(response.encode(), address)
```

Once the node has the IP and port that THOR is running on, it may "register" with the Hub. It does this by making a GET request to `/api/node/register/:nodeId`. The Hub will respond with a payload like the following:

```
{
  "mqtt": {
    "host": "192.168.0.1",
    "password": "hunter2",
    "port": 1883,
    "username": "service"
  },
  "timestamp": 1735927224.037687 // in seconds
}
```

The node can then use this to send and receive data using MQTT, as by default Mosquitto is configured to require authentication.

Currently, registering doesn't do anything for the user, but at the moment the Hub does log which nodes have registered, and when they last registered, which could be used in the future.

2.2.5 System Requirements

The hub should be able to run fully on lower-powered hardware with a relatively recent Linux distribution, such as a Raspberry Pi or an old desktop, but it should work on anything running Python 3.10 or newer. It should work entirely within your local network, and does not require internet access after

the initial setup, however there are some optional features implemented that allows data from the internet to be used.

My deployment of the Hub runs on a Raspberry Pi 3B running Debian 12, using Gunicorn as the HTTP server and Mosquitto as the MQTT broker. I use Gunicorn and Mosquitto because they are the standard for their respective use-cases.

2.2.6 MQTT Bridge

The MQTT module I use for Python, `paho.mqtt`, does not play nicely with the same application running in multiple processes, which is what I have configured in the Gunicorn config for the hub for performance reasons. For this reason, I spun handling MQTT off to its own script, like UDP Autodiscovery. The MQTT bridge isn't that complicated, it simply connects to both the Socket.IO server on the hub and the configured MQTT broker and passes messages between them.

2.3 Lightning Node

The lightning node is the second most important part of this system, behind the Hub, of course. It is a physical device, comprising of three main components: a DFRobot Gravity Lightning Sensor (containing an AMS AS3935 Franklin Lightning Sensor IC), an Arduino Uno and an Arduino Nano ESP32².

I chose the DFRobot sensor because it was relatively inexpensive (about €20) and it claimed to detect lightning from up to 40km away and it supported I²C. I did look into other systems, but most of the ones intended for embedding in a system uses the same lightning sensor anyway, and products aimed at consumers tended to be relatively expensive, such as a US-based company named TALOS selling exclusively lightning sensors, none of which cost less than US\$100, they probably used the same chip anyway³, and I wouldn't be able to extract data from them anyway.

The two microcontrollers serve two different purposes. The Arduino Uno interacts with the lightning sensor over the I²C protocol, and listens for an interrupt signal from the sensor. When this happens, it reads the data from the sensor: it either interprets a lightning signal or the sensor reporting that there is too much noise.

The TX and RX pins on the Uno are connected to the RX and TX pins on the Nano ESP32 respectively, allowing them to communicate over serial using UART. When the Uno needs to send data to the Nano, it will start the line with "~", as the Uno sends some debug information over UART as well to help me troubleshoot the program.

²This can be swapped out for any other MicroPython-compatible microcontroller with Wi-Fi.

³They claim the sensor can detect lightning from up to 25 miles away (about 40km).[8], which is suspiciously similar to the AS3935.

If there's lightning to report, the Uno will print the distance over UART, as well as the "energy" reported by the sensor. I'm not sure what it reports so I'm not logging it anywhere, but I'm leaving it in the Uno's code in case I want to use it in the future.

```
uint8_t lightningDistKm = lightning0.getLightningDistKm();
uint32_t lightningEnergyVal =
    lightning0.getStrikeEnergyRaw();

Serial.print("~LIGHTNING;DISTANCE:");
Serial.print(lightningDistKm);
Serial.print(";ENERGY:");
Serial.print(lightningEnergyVal);
Serial.println("");

// Example output: ~LIGHTNING;DISTANCE:10;ENERGY:0
```

In the case of the noise being too high, it simply returns ~NOISY.

The code running on the Nano is slightly more complicated, as it needs to listen to the UART pins and also send data over the network to the MQTT broker. Instead of using the Arduino language, it uses MicroPython, which is a port of Python on microcontrollers like this. Using MicroPython allows the code to easily be adapted to other microcontrollers.

Once it gets the IP of the Hub and the MQTT credentials (see subsection 2.2.4) and initializes the MQTT client, the Nano goes into a loop where it will wait for a line of data over UART, attempt to decode the data and check if it starts with ~. It will then check if its a notice that the noise is too high, it will send that over MQTT and restart the loop. Otherwise, it parses the distance of the lightning and sends that over MQTT.

You may have noticed that I'm using two similar microcontrollers, and that I could probably take the Arduino Uno out of this system entirely. Unfortunately, the DFRobot sensor refuses to connect over I² to either the Arduino Nano or the Raspberry Pi Pico W I originally got for this project for some reason, but it would connect to the Uno, which is why the Nano receives the data over serial to then broadcast on MQTT.

2.3.1 Testing

When I was working on the lightning node, I tested the latency between Nano sending the lightning event and the Alerts App (section 2.4), with the Nano on Wi-Fi, and an Ethernet connection between the router, the Hub and the computer running Alerts. Over six tests (see Figure 2.3), I found the average latency was 739ms, which is more than low enough for an event like lightning.

Due to circumstances outside of my control, I was unable to test the accuracy of the lightning sensor.

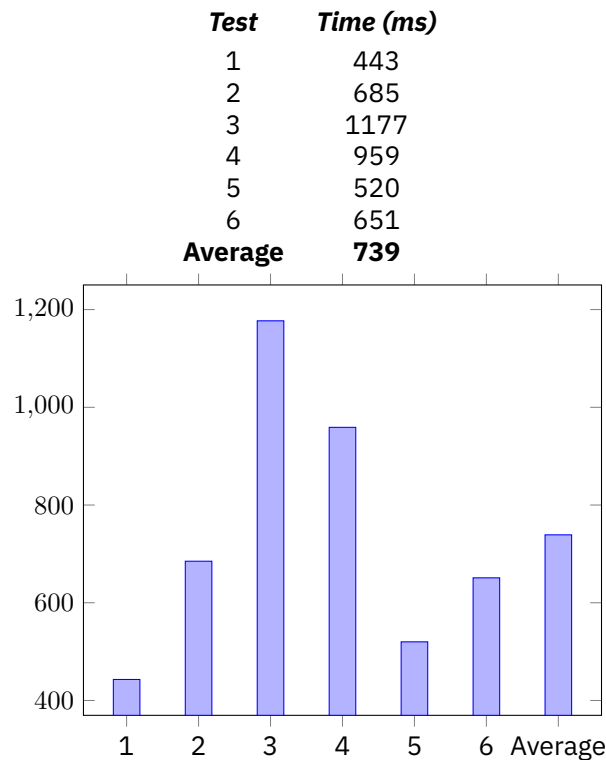


Figure 2.3: Latency of alerts being received in milliseconds

2.4 Alerts App

The Alerts app is a receiver node that runs on a computer. I wrote it in Python as I was already using it for the other components of this project, and I used wxPython as the GUI framework. I decided to use wxPython for various reasons, the main one being that it makes it easy to write apps that act consistently across Windows, GNU/Linux (Figure 2.4a) and macOS (Figure 2.4b) without needed to know the intricacies of a specific platform. It also makes system tray⁴ icons very easy to implement and change and I was already very mildly familiar with it before starting work on THOR.

The Alerts app is very simple. It connects to THOR using Socket.IO and listens for the weather and alerts messages. When weather message arrive, it sets the system tray icon to an icon that matches the current weather. When alerts are sent out, it uses the desktop-notifier module to display a notification.

⁴Or in the case of macOS, the Menu Bar.

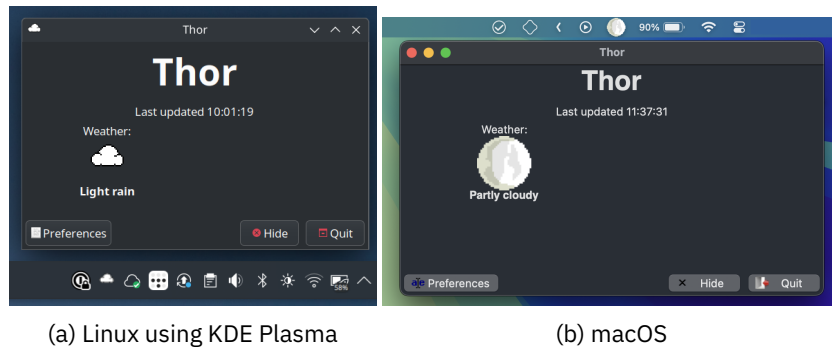


Figure 2.4: THOR Alerts running on various operating systems

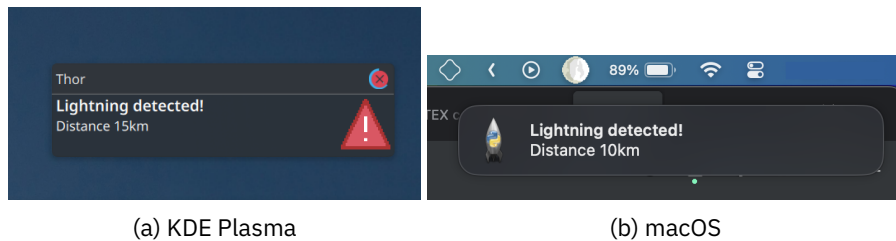


Figure 2.5: An notification showing an alert from THOR.

2.4.1 THORICONS

Icon sets across various systems vary wildly, especially between Linux desktop environments. This means that I would have to maintain a list of mappings from the symbol names given by MET Norway not only to Font Awesome like I previously mentioned the Hub doing, but I would also have to maintain similar lists for Microsoft's Fluent icon set, Apple's SF Symbols and the Freedesktop icon specifications (Linux), which (and this can *especially* be the case for Freedesktop-compliant icon sets) can vary wildly.

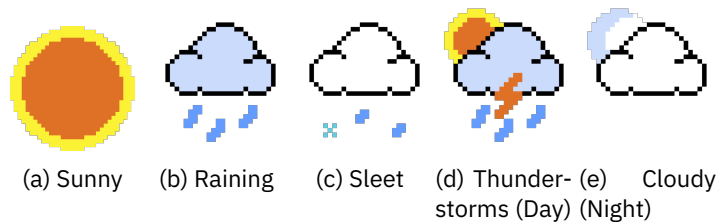


Figure 2.6: Examples of icons in the THORICON set.

For that reason I decided to create a set of weather icons for THOR, as I am experienced in making simple pixel art drawings. The icons are made of

various "parts", which are then merged into various icons by a Python script. THORICON names follow a naming convention of part1-part2-part3, which tells the script how to generate the icon. For example, when given the icon sun-cloud-rain-lightning, the script will paste sun.png onto the image, and then cloud.png, and so on until it has pasted all the parts in the image. It then saves the icon to sun-cloud-rain-lightning.png. I didn't have time to create icons for every symbol name MET Norway returns in their API, but thanks to how I generate the icons adding new icons is quick and painless.

To allow users to choose what icons they prefer on their system, THOR Alerts has a toggle in its preferences to switch between the Freedesktop and THORICON set on Linux. In the future I wish to add support for Mac and Windows icon sets directly, but I use Linux on my main computer so this is difficult to test.

2.5 Display Node



(a) No alerts

(b) A lightning alert on the node



(c) Bugged weather warnings from Met Éireann

Figure 2.7: Pictures of the display node

The "display node" is another receiver node, but this time as a physical device using MQTT. It is a Raspberry Pi Zero 2 W⁵ with a red/black/white Pi-

⁵Though the code and other hardware might work with the original Zero W

moroni Inky pHAT e-Paper display. It is also running a Python script, using the pillow image generation library to generate images to show on the pHAT.

I had to create an image rendering library using pillow because I found myself repeating the same code for drawing text, icons, etc. The script should work with all Inky models, even those with larger screen sizes, because positions and sizes are relative to the screen size. The script is designed for 3-colour models (red, black and white in this case) but it should work with the Inky Impression, which has 7 colours.

This node can fit up to two alerts on screen (see Figure 2.7c) but other screens can fit more alerts. If an alert is severe at all, like the Yellow warnings are in my image, the node can set the colour to something like red to show severity. More severe alerts, like Orange or Red weather warnings, will show a fully red background instead of just the outline.

The main advantage of e-Paper is the low power usage. e-Paper requires very little energy, only using energy when it needs to change the content on screen[9] and using no energy when it doesn't, meaning that the image will stay even if there's no power to the screen at all. It does have a couple of drawbacks: the screen can take time to refresh (the Inky model I'm using takes about 15 seconds to refresh, though apparently the black-and-white model takes about 2 seconds[7]) and colours don't look that great on them yet.

Unfortunately, I didn't have time to 3D print a case for the display node, nor did I have time to make the display node play audio to go with incoming alerts. These are things I would like to look into in the future.

Chapter 3

Conclusion

I'm happy with how THOR turned out, and I learned a lot about Python, various programming concepts, using MQTT and Socket.IO, web programming, hardware programming, I²C, UART, and drawing (see subsection 2.4.1), among a lot of other things, and I developed skills in time management, learning how to use unfamiliar programs/libraries (like $\text{\LaTeX}$ and pillow), and I think I came out of this a better person than I was when I started. Because I didn't use AI at all for this project (not to challenge myself, but for ethical reasons), I feel I really understand how my project works and it makes me very proud of it.

As for the future of THOR, I think there's a lot of potential for the project should I choose to continue it. I intentionally designed it in such a way that allows me or anyone else to add sensors easily with programming knowledge. I'd like to make it easier to add more sensor types. I also want to learn 3D printing so I can make enclosures for the various nodes. I hope to make THOR a system one can easily and inexpensively install in their home, and I'd love if it could hook into other smart home systems like Home Assistant¹. I'd also like to be able to make alerts appear on mobile. I didn't do it this time because the Web Notifications API for JavaScript only works in secure contexts (HTTPS), which is inconvenient over a local network, and I don't have the skills, the money or the hardware needed to make an Android *and* an iOS app.

3.1 Acknowledgments

I would like to thank the maintainers of every library, module, package and tool I used to help make THOR. I would also like to thank the following people in my personal life:

Ms Abbott If it weren't for Ms Abbott, my Science teacher through Junior Cert, I probably wouldn't be at BTYSTE. She has always been encouraging of my scientific endeavors, no matter how silly (though apparently fire and

¹Home Assistant already supports MQTT, so maybe it's possible. I never tested it.

explosions are "unsafe" and cross the line) or how serious. She is probably the greatest teacher I've had so far, and maybe she'll be the greatest I'll ever have. I'm very thankful to have her as my teacher and mentor.

Met Éireann, MET Norway and every other meteorological service around the world

I'd like to thank Met Éireann and MET Norway for providing their data free of charge online. This is a tremendous help to me and anyone else who wishes to build something based on weather data. I'd also like to thank *every* meteorological service for doing the thankless work that they do monitoring our planet.

My parents I'd like to thank my parents, Roibeard and Celia, for supporting me and giving me so much time to work on my project, giving me feedback on the name, and for continuing to help me through life no matter how tough it gets.

References

- [1] Met Éireann. *Description of Met Éireann Warnings rss feed*. 2020. URL: https://www.met.ie/Open_Data/Warnings/Met_Eireann_Warning_description_June2020.pdf.
- [2] Met Éireann. *Lightning Detection*. URL: <https://www.met.ie/science/valentia/lightning-detection>.
- [3] NOAA. *Frequently Asked Questions About Lightning*. URL: <https://www.nssl.noaa.gov/education/svrwx101/lightning/faq/>.
- [4] MET Norway. *Frequently Asked Questions*. URL: <https://api.met.no/doc/FAQ>.
- [5] National Oceanic and Atmospheric Association. *Severe Weather 101 – Lightning Detection*. URL: <https://www.nssl.noaa.gov/education/svrwx101/lightning/detection/>.
- [6] ScioSense. *AS3935 Franklin Lightning Sensor IC Datasheet*. URL: <https://www.sciosense.com/wp-content/uploads/2023/12/AS3935-Data-Sheet.pdf>.
- [7] Pimoroni Store. *Inky pHAT (ePaper/eInk/EPD)*. URL: <https://shop.pimoroni.com/products/inky-phat?variant=12549254217811>.
- [8] TALOS. *TALOS Lightning Detectors. This is the state-of-the-art lightning detector*. URL: <https://taloslightningdetectors.com/product/talos-lightning-detectors/>.
- [9] u/shogzilla. *r/eink Energy consumption of eink displays?* URL: https://old.reddit.com/r/eink/comments/kra78t/energy_consumption_of_eink_displays/gi9pxvw/.

100% human-made. No Generative AI was knowingly used in the creation of this booklet or this project.